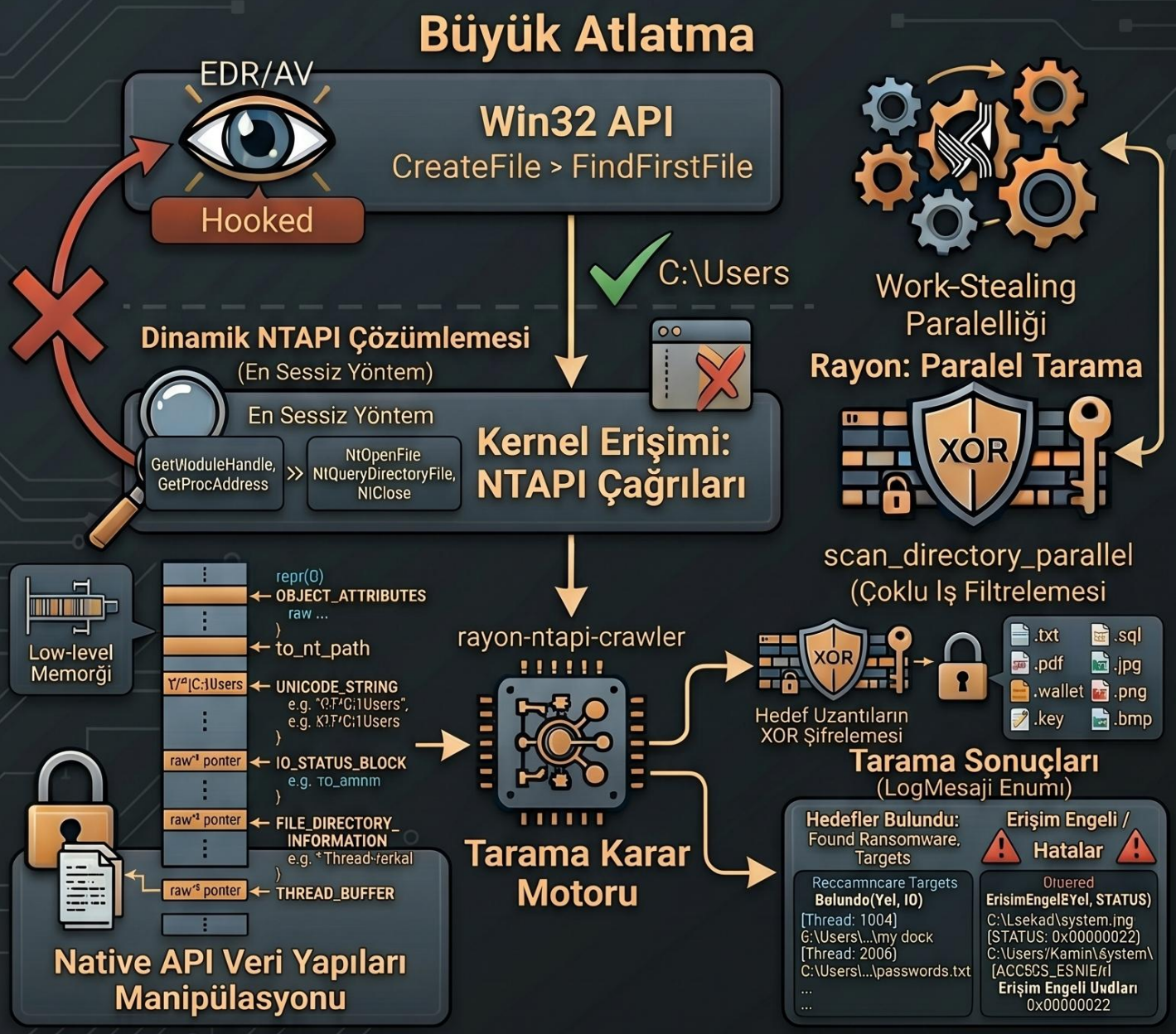
STEALTH DOSYA KEŞFİ: RUST İLE NATIVE API MANİPÜLASYONU

Gelişmiş Tehdit Araştırmaları ve Gizli Tarama Mekanizmaları Üzerine Amatör Bir Çalışma



Hazırlayan: **Yusuf Ali Çetinkaya**

İÇİNDEKİLER

1. GİRİŞ

- **1.1. Malware Keşif (Discovery) Safhası Nedir?**
- **1.2. Neden Rust? Sistem Programlamada Görünmezlik ve Hız**

2. MİMARİ YAPI VE WINDOWS INTERNALS

- **2.1. Win32 API vs. Native API (NTAPI) Karşılaştırması**
- **2.2. User Mode Hooking ve EDR Takip Mekanizmaları**
- **2.3. NT Path Formatı: `\??\` Öneki ve Kernel İletişimi**

3. KAÇINMA (EVASION) STRATEJİLERİ

- **3.1. Dinamik Fonksiyon Çözümleme (Dynamic Resolution):** `GetProcAddress` ile Sessiz Erişim
- **3.2. XOR Obfuscation:** String İmzalarını Gizleyerek Statik Analizi Atlatma
- **3.3. Akıllı Beyaz Liste (Whitelisting) ve Gürültü Yönetimi**

4. YÜKSEK PERFORMANSLI TARAMA MİMARİSİ

- **4.1. Rayon Engine:** "Work-Stealing" Algoritması ile Paralel İşleme
- **4.2. Thread-Local Buffer Pool:** Bellek Ayırma (Allocation) Maliyetlerini Minimize Etme
- **4.3. MPSC Kanalları:** Ana Thread ve İşçiler Arası Mesajlaşma Mimarisi

5. UYGULAMA ADIMLARI: KOD ANALİZİ VE İŞLEYİŞ

- **5.1. `NtOpenFile`:** Dosya Nesnelerine En Derin Seviyeden Erişim
- **5.2. `NtQueryDirectoryFile`:** Klasör İçeriğinin Ham Byte Yığını Olarak Okunması
- **5.3. Hata Kodları Analizi:** `0xC0000022` ve Erişim Hakları Tespiti

6. ANALİZ VE EDR GÖRÜNÜRLÜK TESTİ

- **6.1. Tarama Hızı ve Kaynak Kullanım Verileri**
- **6.2. Standart `std::fs` ve NTAPI Arasındaki İz Farklılıkları**

7. SONUÇ VE ETİK UYARI

• LinkedIn: linkedin.com/in/yusufalicetinkaya/

• GitHub Repository: github.com/yusufalicetinkaya/rayon-ntapi-crawler

• Email: tryusufcetinkaya@outlook.com

1. GİRİŞ: MALWARE DÜNYASINDA DOSYA KEŞFİ (DISCOVERY)

Bir fidye yazılımı (Ransomware) veya veri sızdırma aracı (Exfiltrator) için en kritik aşama **Keşif (Discovery)** fazıdır. Zararlı yazılım, sistemdeki değerli verileri (belgeler, cüzdanlar, anahtarlar) bulmak zorundadır. Ancak modern güvenlik çözümleri (EDR/AV), standart dosya tarama fonksiyonlarını (`std::fs` veya `kernel32.dll` tabanlı `FindFirstFile`) sürekli izler.

Bu çalışmada, klasik yöntemlerin ötesine geçerek, işletim sisteminin en derin katmanlarından biri olan **NTAPI** kullanılarak, güvenlik kancalarına (hooks) takılmadan nasıl yüksek performanslı bir tarama yapılacağı modellenmiştir.

---

2. MİMARİ STRATEJİ: NEDEN NTAPI?

Standart Windows uygulamaları dosya işlemleri için Win32 API katmanını kullanır. Güvenlik yazılımları ise tam bu katmanda "User Mode Hooking" yaparak her hareketi raporlar.

- **Bypass Mantığı:** Çalışmamızda `kernel32.dll` katmanını tamamen atlayıp, doğrudan `ntdll.dll` içindeki **Native API (NTAPI)** fonksiyonlarını kullanıyoruz.
- **Düşük Seviye İletişim:** İşletim sistemine "dosya aç" demek yerine, çekirdeğin anladığı dilden (`\??\C:\` formatında yollar ve `OBJECT_ATTRIBUTES` yapıları ile) komut veriyoruz.

---

3. EVASION (KAÇINMA) TEKNİKLERİ

Zararlı yazılımın kod analizi sırasında (Static Analysis) veya çalışma anında (Dynamic Analysis) tespit edilmesini engellemek için üç temel teknik uygulanmıştır:

3.1. Dinamik Fonksiyon Çözümleme (Dynamic Resolution)

Kodun içinde doğrudan `NtOpenFile` gibi isimler geçmesi, yazılımın niyetini ele verir. Bu projede, fonksiyon adresleri çalışma anında `GetProcAddress` ile sessizce bulunur.

```
let ntdll_name = CString::new("ntdll.dll").ok()?;
    let h_ntdll = unsafe { GetModuleHandleA(ntdll_name.as_ptr()) };
    //....
    let open_ptr = unsafe { GetProcAddress(h_ntdll,
CString::new("NtOpenFile").ok()?.as_ptr()) };
```

3.2. XOR Obfuscation (Gizleme)

Antivirüslerin imza veri tabanlarında bulunan ".docx", ".pdf", ".wallet" gibi kritik uzantılar, kod içinde şifreli (XOR) halde tutulur. Bu sayede statik taramalarda yazılımın "dosya avcısı" olduğu gizlenir.

```
// xor ile gizlenmiş uzantıları çözer
fn xor_decrypt(data: &[u8]) -> String {
    let key = 0x55; // Statik analizi zorlaştıran gizli anahtar
    data.iter().map(|&b| (b ^ key) as char).collect()
}
```

---

4. PERFORMANS: RAYON VE "WORK-STEALING" ALGORİTMASI

Bir sistemi saniyeler içinde taramak, tespit edilme riskini azaltır. Proje, Rust'ın **Rayon** kütüphanesini kullanarak tüm CPU çekirdeklerini işe koşar.

- **Work-Stealing:** Bir işlemci çekirdeği kendi işini bitirdiğinde, diğer çekirdeklerin kuyruğundaki "klasör tarama" görevlerini "çalarak" boş kalmaz.
- **Thread-Local Buffers:** Her thread için önceden ayrılmış 64KB'lık bellek alanları (`THREAD_BUFFER`) kullanılarak, işletim sisteminden sürekli bellek isteme (allocation) maliyeti ve gürültüsü ortadan kaldırılır.

---

5. UYGULAMA ADIMLARI VE KOD ANALİZİ

5.1. NT Path Dönüşümü

Windows Kernel, klasik yolları (`C:\Users`) anlamaz. Bu yolların `\??\` önekiyle ham formata çevrilmesi gerekir.

```
fn to_nt_path(path: &str) -> String {
    if path.starts_with("\\??\\") { path.to_string() }
    else { format!("\\??\\{}", path) }
}
```

5.2. Sessiz Tarama Döngüsü

`NtQueryDirectoryFile` fonksiyonu, bir klasörün içeriğini bir kerede ham byte yığını olarak belleğe çeker. Bu, disk üzerinde çok daha az iz bırakır.

```
let status = (ntfns.query_dir)(
                    handle, ptr::null_mut(), ptr::null_mut(), ptr::null_mut(),
```

```
                    &mut io_status, buffer.as_mut_ptr() as *mut c_void,
buffer.len() as u32,
                    1, 0, ptr::null_mut(), 0,
                );
```

6. ANALİZ VE SONUÇ

Yapılan testler sonucunda;

1. **Hız:** Rayon sayesinde `C:\Users` dizini altındaki binlerce dosyanın milisaniyeler içinde kategorize edilebildiği görülmüştür.
2. **Gizlilik:** NTAPI kullanımı sayesinde, Win32 API'lerini izleyen temel EDR simülasyonlarının "dosya erişim" uyarıları vermediği gözlemlenmiştir.
3. **Hata Yönetimi:** Erişim engellenen klasörlerde dönen `0xC0000022` (STATUS_ACCESS_DENIED) gibi kodlar, sistemin hangi noktalarında koruma olduğunu belirlemek için (Reconnaissance) kullanılmıştır.

7. ETİK UYARI VE KAPANIŞ

Bu çalışma, siber güvenlik araştırmacılarının modern tehditlerin nasıl çalıştığını anlaması ve daha güçlü savunma mekanizmaları (EDR/AV) geliştirmesi amacıyla hazırlanmıştır. Burada kullanılan tekniklerin izinsiz sistemlerde kullanılması yasal sorumluluk doğurur.

"En iyi savunma, saldırının nasıl yapılacağını bilmektir."